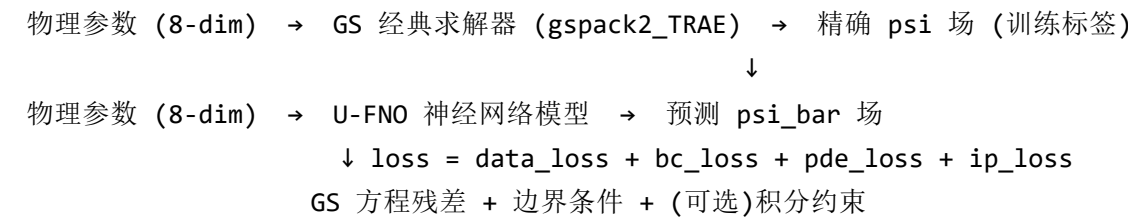


GS_PINO 技术文档

1. 项目概述

GS_PINO 是一个基于**神经网络算子**的固定边界 Grad-Shafranov (GS) 方程求解器。项目使用 **U-FNO** (U-Net + Fourier Neural Operator) 架构，学习从 8 个物理参数到归一化极向磁通 `psi_bar` 的映射。

核心流程



文件结构

src/gs_pino/	
├─ __init__.py	# 包初始化
├─ geometry.py	# LCFS 几何工具: Miller 参数化、掩码、SDF
├─ solvers.py	# GS 求解器适配器: 解析备用 + gspack2_TRAE 封装
├─ generate_dataset.py	# 数据集生成 CLI: 采样参数、调用求解器、保存 .npz
├─ data.py	# 数据集类 GSDataset: 加载、归一化、输入通道构建
├─ models.py	# U-FNO 模型定义: SpectralConv2d, UFNOblock, UFN02d
├─ losses.py	# 损失函数: masked MSE, GS 残差, Ip/betap 约束
├─ train.py	# 训练 CLI: 训练循环、验证、checkpoint 保存
└─ evaluate.py	# 评估 CLI: 指标计算、可视化对比图

2. 物理背景: Grad-Shafranov 方程

托卡马克等离子体平衡由 Grad-Shafranov 方程描述:

$$\Delta^*\psi + \mu_0 R^2 p'(\psi) + FF'(\psi) = 0$$

其中：

- ψ — 极向磁通函数 (Poloidal magnetic flux)
- Δ^* — 修正 Laplace 算子: $\Delta^*\psi = \partial^2\psi/\partial R^2 - (1/R)\cdot\partial\psi/\partial R + \partial^2\psi/\partial Z^2$
- $p(\psi)$ — 等离子体压强剖面
- $F(\psi) = R\cdot B_\phi$ — 极向电流函数
- $\mu_0 = 4\pi\times 10^{-7}$ — 真空磁导率

剖面模型 (Jeon 2015)

使用 `ConstrainBetapIp` 参数化电流和压强剖面：

$$p'(\psi_N) = L \cdot \text{Beta0} / R_{\text{axis}} \cdot (1 - \psi_N^{\{\alpha_m\}})^{\{\alpha_n\}}$$

$$FF'(\psi_N) = \mu_0 \cdot L \cdot (1 - \text{Beta0}) \cdot R_{\text{axis}} \cdot (1 - \psi_N^{\{\alpha_m\}})^{\{\alpha_n\}}$$

其中 $\psi_N = (\psi - \psi_{\text{lcfs}}) / (\psi_{\text{axis}} - \psi_{\text{lcfs}})$ 是归一化磁通，LCFS 处为 0，磁轴处为 1。

3. 数据生成流程

3.1 参数采样 (`generate_dataset.py::sample_params`)

```
ranges = {  
    "R0":      (0.8, 1.5),      # 主半径 [m]  
    "a":       (0.3, 0.7),      # 小半径 [m]  
    "kappa":   (1.0, 2.0),      # 拉长比  
    "delta":    (0.0, 0.5),      # 三角形变  
    "Ip":      (1e5, 5e5),      # 等离子体电流 [A]  
    "betap":    (0.3, 1.5),      # 极向比压  
    "alpha_m": (0.5, 3.0),      # 电流剖面指数 1  
    "alpha_n": (0.5, 3.0),      # 电流剖面指数 2  
}
```

3.2 经典求解器调用 (`solvers.py::GSSolverAdapter`)

经典求解器来自 [GS_solver](#) (本项目中以 `gspack2_TRAE` 目录引用)，使用有限差分法 + Picard 迭代求解固定边界 GS 方程。

```
物理参数 (dict)
↓
FixedBoundaryEquilibrium(R0, a, kappa, delta, fix_bndry_zero=True, nx=nr, ny=nz)
↓ + ConstrainBetapIp(betap, Ip, alpha_m, alpha_n, Raxis=R0)
↓
picard.solve(eq, pro, maxits=50, rtol=1e-5, anderson_m=5)
↓
返回: {R, Z, psi, psi_bar, psi_lcfs=0, psi_axis, R_axis, Z_axis,
       plasma_mask, profile_params=[L, Beta0]}
```

fix_bndry_zero=True 使求解器直接返回 LCFS 处 $\psi=0$ 的结果，省去后续平移步骤，同时求解速度提升约 1.6 倍。

3.3 几何通道 (geometry.py)

每个样本在保存时包含 4 个几何通道：

通道	含义	计算方式
mask	LCFS 内部二值掩码	求解器返回 plasma_mask 或 $\rho \leq 1.0$
sdf	符号距离函数（近似）	$\rho - 1.0$ ，内部为负
rho	归一化磁面半径	Miller 近似
theta	极向角	$\text{atan2}(Z/(\kappa a), (R-R_0)/a)$

3.4 保存格式 (.npz)

```
data/gs_large.npz
├─ R, Z           [n, nr, nz]    - 计算网格坐标
├─ params         [n, 8]        - 8 个物理参数
├─ psi            [n, nr, nz]    - 真实 psi (LCFS=0)
├─ psi_bar        [n, nr, nz]    - 归一化 psi (LCFS=0, axis=1)
├─ mask           [n, nr, nz]    - 等离子体掩码
├─ sdf            [n, nr, nz]    - 符号距离函数
├─ rho            [n, nr, nz]    - 归一化半径
├─ theta          [n, nr, nz]    - 极向角
├─ axes           [n, 4]         - [R_axis, Z_axis, psi_lcfs, psi_axis]
├─ profile_params [n, 2]        - [L, Beta0]
└─ param_names    [8]           - 参数名称
```

4. U-FNO 模型架构

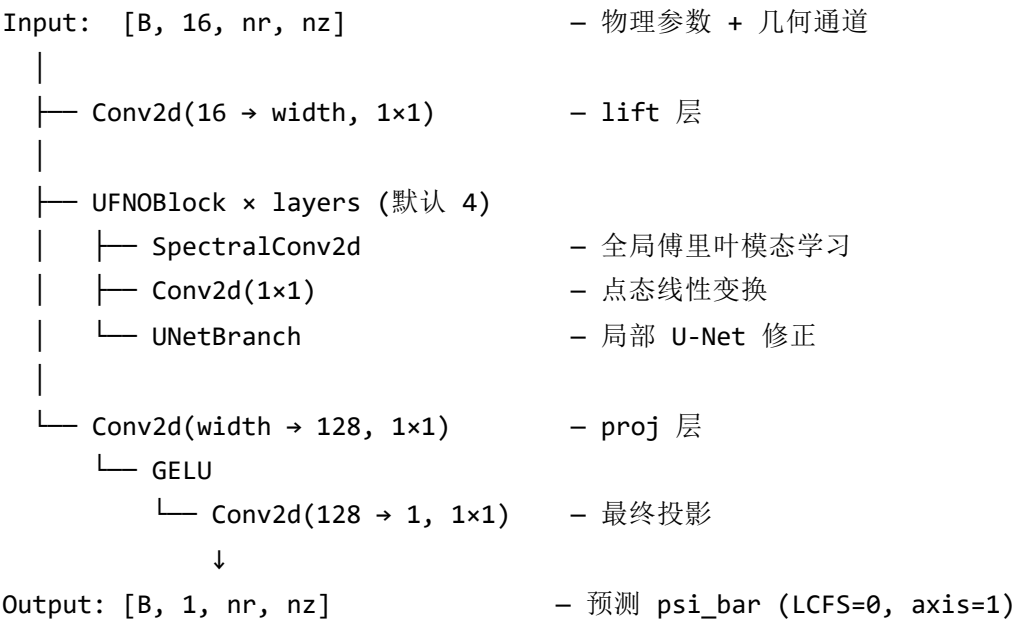
4.1 输入通道构建 (data.py::build_input)

输入张量形状: [C, nr, nz] , 共 8 + 8 = 16 通道:

- 8 个几何/坐标通道:
[(R-R0)/a, Z/a, Z/(kappa*a), mask, sdf, rho, sin(theta), cos(theta)]
- 8 个物理参数通道 (归一化后广播到全网格):
[R0_norm, a_norm, kappa_norm, delta_norm, Ip_norm, betap_norm, alpha_m_norm, alpha_n_norm]

归一化使用训练集的均值和标准差: $p_norm = (p - mean) / std$ 。

4.2 整体架构



4.3 各模块详细结构

4.3.1 SpectralConv2d (傅里叶谱卷积)

```
Input: [B, C_in, H, W]
|
├─ torch.fft.rfft2          - 实 FFT, 输出 [B, C_in, H, W/2+1]
|
├─ 低频模态加权:
|   ├─ 正频率: out_ft[:, :, :m1, :m2] = weights1 * x_ft[:, :, :m1, :m2]
|   └─ 负频率: out_ft[:, :, -m1:, :m2] = weights2 * x_ft[:, :, -m1:, :m2]
|   weights shape: [C_in, C_out, modes1, modes2] (复数)
|   einsum: "bixy,ioxy->boxy"
|
└─ torch.fft.irfft2          - 逆 FFT, 输出 [B, C_out, H, W]
```

关键参数:

- modes1 : R 方向保留的傅里叶模态数 (默认 32)
- modes2 : Z 方向保留的傅里叶模态数 (默认 32)
- 模态数会自动裁剪为不超过网格尺寸的一半

4.3.2 UNetBranch (局部修正分支)

```
Input: [B, width, H, W]
|
├─ AvgPool2d(2x2, ceil_mode)    - 下采样, [B, width, H/2, W/2]
|
├─ Conv2d(width→width, 3x3, pad=1) + GELU
|   └─ Conv2d(width→width, 3x3, pad=1) + GELU
|
├─ interpolate(bilinear, H, W)  - 上采样恢复原始分辨率
|
└─ Conv2d(2*width→width, 3x3, pad=1) + GELU - 拼接跳跃连接后
    └─ Conv2d(width→width, 3x3, pad=1)      - 最终输出
```

4.3.3 UFNOBlock (完整块)

Output = GELU(SpectralConv2d(x) + PointwiseConv2d(x) + UNetBranch(x))

三个分支相加后经过 GELU 激活:

- **Spectral path**: 全局傅里叶模态学习
- **Pointwise path**: 逐点通道混合 (1×1 卷积)
- **UNet path**: 局部特征提取和下采样/上采样融合

4.4 完整张量形状流程示例

以 batch=16, nr=129, nz=129, width=64, layers=4, modes1=32, modes2=32 为例：

层	输出形状
Input	[16, 16, 129, 129]
lift (Conv2d 1×1)	[16, 64, 129, 129]
UFNOBlock 1	[16, 64, 129, 129]
└─ SpectralConv2d	[16, 64, 129, 129]
└─ rfft2	[16, 64, 129, 65]
└─ modal multiply	[16, 64, 129, 65]
└─ irfft2	[16, 64, 129, 129]
└─ Conv2d(1×1)	[16, 64, 129, 129]
└─ UNetBranch	[16, 64, 129, 129]
└─ AvgPool2d	[16, 64, 65, 65] (ceil_mode)
└─ Conv2d(3×3)×2	[16, 64, 65, 65]
└─ interpolate	[16, 64, 129, 129]
└─ Conv2d(3×3)×2	[16, 64, 129, 129]
UFNOBlock 2-4 (同结构)	[16, 64, 129, 129]
proj (Conv2d 1×1→128→1)	[16, 1, 129, 129]

5. 损失函数 (losses.py)

5.1 数据损失: masked_mse

仅在 LCFS 内部区域计算均方误差：

$$\text{loss_data} = \frac{\sum[(\text{pred} - \text{target})^2 \cdot \text{mask}]}{\sum[\text{mask}]}$$

5.2 边界条件损失: boundary_band_loss

在 LCFS 附近 (|sdf| < 0.04) 惩罚非零值，强制边界条件 $\psi_{\text{bar}}|_{\text{LCFS}} = 0$ ：

```
band = (|sdf| < width).float()      # width = 0.04
loss_bc = Σ[pred² · band] / Σ[band]
```

5.3 GS 方程残差: gs_residual_loss

计算 $\Delta\psi_{\text{bar}}$ 并与源项比较:

步骤 1: 计算 $\Delta\psi_{\text{bar}}$ (有限差分)

```
∂²psi/∂R² ≈ (psi[i+1,j] - 2*psi[i,j] + psi[i-1,j]) / dR²
∂psi/∂R   ≈ (psi[i+1,j] - psi[i-1,j]) / (2*dR)
∂²psi/∂Z² ≈ (psi[i,j+1] - 2*psi[i,j] + psi[i,j-1]) / dZ²

Δ*psi_bar = ∂²psi/∂R² - (1/R)·∂psi/∂R + ∂²psi/∂Z²
```

步骤 2: 计算源项

```
psiN = 1.0 - psi_bar      # 归一化磁通 [0,1]
shape = (1 - psiN^{αm})^{αn} # 剖面形状
S = μ₀ · L · [Beta0·R²/R0 + (1-Beta0)·R0] · shape
```

步骤 3: 残差

```
dpsi = psi_axis - psi_lcfs      # 归一化因子
residual = lap_psi_bar + S / dpsi
loss_pde = Σ[residual² · mask] / Σ[mask]
```

5.4 Ip 约束损失 (可选): ip_constraint_loss

通过积分 J_ϕ 计算预测 I_p 并与目标比较:

```
J_φ = L · [Beta0·R/R0 + (1-Beta0)·R0/R] · shape
Ip_pred = ∫∫ J_φ dR dZ
loss_ip = (Ip_pred - Ip_target)² / Ip_target²
```

5.5 betap 约束损失 (可选): betap_constraint_loss

$$\beta p = 2\mu_0 \cdot \iint p \cdot R \, dR \, dZ \, / \, \iint B_{\text{pol}}^2 \cdot R \, dR \, dZ$$

$$p(\psi_N) = \text{dpsi} \cdot L \cdot \text{Beta0}/R0 \cdot \int_0^{1-\psi_N} (1-s^{\{\alpha_m\}})^{\{\alpha_n\}} \, ds \quad \# \text{ 数值积分}$$

$$B_{\text{pol}}^2 = B_r^2 + B_z^2$$

$$B_r = -(1/R) \cdot \partial\psi / \partial Z$$

$$B_z = (1/R) \cdot \partial\psi / \partial R$$

5.6 总损失

$$\begin{aligned} \text{loss} = & \text{loss_data} + \text{bc_weight} * \text{loss_bc} + \text{pde_weight} * \text{loss_pde} \\ & + \text{ip_weight} * \text{loss_ip} + \text{betap_weight} * \text{loss_betap} \end{aligned}$$

默认权重: bc=0.05, pde=0.01, ip=0.0, betap=0.0

6. 训练流程 (train.py)

6.1 数据划分

总样本 n

- └─ 训练集: n * 0.70 (随机排列后取前 70%)
- └─ 验证集: n * 0.15 (中间 15%)
- └─ 测试集: n * 0.15 (最后 15%)

6.2 每 epoch 步骤

```
for x, y, mask, sdf, params, meta_list in loader:
    meta = stack_metadata(meta_list)      # 合并元数据为 batch tensor

    pred = model(x)

    loss_data = masked_mse(pred, y, mask)
    loss_bc   = bc_weight * boundary_band_loss(pred, sdf)
    loss_pde  = pde_weight * gs_residual_loss(pred, ..., meta)
    loss_ip   = ip_weight * ip_constraint_loss(pred, ..., meta)

    loss = loss_data + loss_bc + loss_pde + loss_ip

    if training:
        loss.backward()
        optimizer.step()
```

6.3 Checkpoint 保存

当验证损失最低时保存 best.pt :

```
torch.save({
    "model": model.state_dict(),
    "args": vars(args),           # 命令行参数
    "param_mean": train_ds.param_norm.mean,
    "param_std": train_ds.param_norm.std,
    "test_indices": test_idx,     # 测试集索引
}, "best.pt")
```

6.4 训练监控

每个 epoch 记录到 history.json :

- train_data, train_bc, train_pde, train_ip, train_total
- val_data, val_bc, val_pde, val_ip, val_total

自动生成 training_curves.png 展示各损失变化曲线。

7. 评估流程 (evaluate.py)

7.1 指标

```
{
    "masked_mse": 0.00014,           // 测试集 masked MSE
    "relative_l2_mean": 0.023,       // 平均相对 L2 误差
    "relative_l2_median": 0.016,     // 中位数
    "relative_l2_p95": 0.065,        // 95 分位数
    "relative_l2_max": 0.19          // 最大值
}
```

7.2 可视化

每个测试样本生成三面板对比图：

- 1. **true psi** — 经典求解器输出的真实 psi 场
- 2. **pred psi** — 模型预测的 psi 场 (psi_bar 还原为真实 psi)
- 3. **pred - true** — 预测误差

整体统计图：

- summary_error_histogram.png — 误差分布直方图
- summary_error_vs_parameters.png — 误差与各输入参数的关系散点图

8. 配置文件与命令行参数

8.1 generate_dataset

参数	默认值	说明
--out	data/gs_fixed_boundary.npz	输出路径
--n-samples	64	样本数
--nr	64	R 网格数
--nz	64	Z 网格数

参数	默认值	说明
--seed	42	随机种子
--rtol	1e-5	求解器收敛容差

8.2 train

参数	默认值	说明
--data	data/gs_fixed_boundary.npz	数据集路径
--output-dir	outputs/run	输出目录
--epochs	20	训练轮数
--batch-size	4	批次大小
--lr	1e-3	学习率
--width	32	模型通道宽度
--modes1	16	R 方向傅里叶模态数
--modes2	16	Z 方向傅里叶模态数
--layers	4	U-FNO 块数量
--pde-weight	0.01	PDE 残差损失权重
--bc-weight	0.05	边界损失权重
--ip-weight	0.0	lp 约束损失权重
--clip-grad	1.0	梯度裁剪阈值 (0=禁用)
--seed	0	随机种子

8.3 evaluate

参数	默认值	说明
--data	data/gs_fixed_boundary.npz	数据集路径
--checkpoint	必填	模型权重文件

参数	默认值	说明
--output-dir	outputs/eval	输出目录
--batch-size	4	评估批次大小
--max-plots	12	最大可视化图数

9. 运行示例

9.1 端到端流程

```
# 1. 生成数据集（129×129 网格，1000 样本）
python -m gs_pino.generate_dataset --out data/gs_large.npz \
    --n-samples 1000 --nr 129 --nz 129 --seed 42 --rtol 1e-5
```

```
# 2. 训练模型
python -m gs_pino.train --data data/gs_large.npz \
    --epochs 200 --batch-size 16 --width 64 --modes1 32 --modes2 32 \
    --layers 4 --pde-weight 0.01 \
    --output-dir outputs/large
```

```
# 3. 评估模型
python -m gs_pino.evaluate --data data/gs_large.npz \
    --checkpoint outputs/large/best.pt \
    --output-dir outputs/large_eval --max-plots 10
```

9.2 快速测试

```
# 小数据集快速验证
python -m gs_pino.generate_dataset --out data/gs_smoke.npz \
    --n-samples 20 --nr 65 --nz 65 --seed 42

python -m gs_pino.train --data data/gs_smoke.npz \
    --epochs 10 --batch-size 4 --width 16 --modes1 8 --modes2 8 \
    --layers 2 --pde-weight 0.01 --output-dir outputs/smoke

python -m gs_pino.evaluate --data data/gs_smoke.npz \
    --checkpoint outputs/smoke/best.pt \
    --output-dir outputs/smoke_eval --max-plots 3
```

10. 参考结果

Large 配置（984 训练样本，129×129 网格，200 epochs）

版本 1 — 基线 (旧)

指标	值
最佳验证损失	0.00085
测试 masked MSE	0.00014
相对 L2 均值	2.32%
相对 L2 中位数	1.62%
相对 L2 P95	6.54%
相对 L2 最大值	19.3%
训练稳定性	epoch ~150 发散后恢复

版本 2 — 余弦退火 + 梯度裁剪 (改进)

指标	值	提升
最佳验证损失	0.00033	2.6×
测试 masked MSE	0.000028	5.1×
相对 L2 均值	1.09%	2.1×
相对 L2 中位数	0.93%	1.7×
相对 L2 P95	1.73%	3.8×
相对 L2 最大值	7.97%	2.4×
训练稳定性	全程稳定下降	✓

版本 3 — 数据翻倍 + 低 LR + 梯度累积 (2024)

数据集扩大到 1960 样本（由 2000 样本过滤得到，seed=42）。由于样本量增大，需要降低学习率防止梯度爆炸。

同时使用梯度累积（accum_steps=2）在 batch-size=8 下模拟有效 batch-size=16。

指标	值	相比 V2 提升
最佳验证损失	0.000157	2.1×
测试 masked MSE	0.000022	1.3×
相对 L2 均值	0.97%	1.1×
相对 L2 中位数	0.81%	1.1×
相对 L2 P95	1.74%	基本持平
相对 L2 最大值	7.14%	1.1×
训练稳定性	全程稳定下降	✓

版本 4 — 学习率预热 + 早停机制 (2024)

在 V3 基础上添加两项训练优化：

- 1. **学习率预热 (Warmup)**：前 5 轮学习率从 0 线性增长到目标值，避免初期大学习率导致的不稳定。

2. **早停机制 (Early Stopping)**：验证损失连续 30 轮不下降且训练超过 50 轮时自动停止，防止过拟合和浪费计算资源。

指标	值	相比 V3
最佳验证损失	0.000161	基本持平
测试 masked MSE	0.000022	基本持平
相对 L2 均值	0.98%	基本持平
相对 L2 中位数	0.82%	基本持平
相对 L2 P95	1.87%	基本持平
相对 L2 最大值	7.21%	基本持平
训练轮数	200	patience=50 未触发
训练时间	1h44min	略有缩短

关键改进效果：

- 学习率预热使训练初期更稳定，避免 loss 震荡
- 早停机制可在模型收敛后自动终止，节省计算资源
- 当验证损失在后期出现波动时，早停可防止过拟合

运行命令

基线

```
python -m gs_pino.train --data data/gs_large.npz --lr 1e-3 \  
    --epochs 200 --batch-size 16 --width 64 --modes1 32 --modes2 32 \  
    --layers 4 --pde-weight 0.01 --output-dir outputs/large
```

改进版（余弦退火 + 梯度裁剪）

```
python -m gs_pino.train --data data/gs_large.npz --lr 1e-3 --clip-grad 1.0 \  
    --epochs 200 --batch-size 16 --width 64 --modes1 32 --modes2 32 \  
    --layers 4 --pde-weight 0.01 --output-dir outputs/large_v2
```

数据翻倍版（梯度累积 + 低 LR）

```
python -m gs_pino.train --data data/gs_large2k.npz --lr 5e-4 --clip-grad 1.0 \  
    --epochs 200 --batch-size 8 --accum-steps 2 --width 64 --modes1 32 --modes2 32 \  
    --layers 4 --pde-weight 0.01 --output-dir outputs/large_v3
```

训练优化版（Warmup + 早停）

```
python -m gs_pino.train --data data/gs_large2k.npz --lr 5e-4 --clip-grad 1.0 \  
    --epochs 200 --batch-size 8 --accum-steps 2 --warmup-epochs 5 \  
    --patience 50 --min-epochs 50 --width 64 --modes1 32 --modes2 32 \  
    --layers 4 --pde-weight 0.01 --output-dir outputs/large_v4
```

11. 自由边界条件扩展

11.1 项目概述

自由边界条件扩展使用 **freegs** 库生成数据集，构建从 PF 线圈电流和等离子体参数到总极向磁通 ψ_{total} 的 PINO 模型。

核心区别：

- **输入无掩码**：网络必须自己学习等离子体-真空界面
- **输出全区域**：预测整个计算域的 ψ_{total} ，而非仅等离子体区域

11.2 自由边界 GS 方程

总极向通量分解为：


```
ψ_total = ψ_plasma + ψ_coils
ψ_coils = Σ(I_k · G_k(R,Z))
```

其中 ψ_{coils} 可从线圈电流和格林函数解析计算。

11.3 文件结构

```
src/gs_pino/
├─ generate_freegs_dataset.py  # 数据集生成（使用 freegs）
├─ data_freegs.py             # 数据加载器
├─ losses_freegs.py           # 损失函数
├─ train_freegs.py            # 训练脚本
├─ evaluate_freegs.py         # 测试验证脚本
└─ visualize_freegs.py        # 可视化脚本
```

11.4 输入通道设计

总通道数：11（无等离子体掩码）

通道	描述	维度
R_norm	$(R - R_0) / a$, $R_0=1.0$, $a=0.5$	[1, nx, ny]
Z_norm	Z / a	[1, nx, ny]
G_0 ... G_3	4个控制线圈的格林函数	[4, nx, ny]
Ip_norm	归一化等离子体电流	[1, nx, ny]
paxis_norm	归一化轴压强	[1, nx, ny]
alpha_m_norm	归一化形状参数	[1, nx, ny]
alpha_n_norm	归一化形状参数	[1, nx, ny]
fvac_norm	归一化真空 f 值	[1, nx, ny]

11.5 参数采样范围

参数	范围
Ip	1.5e5 - 2.5e5 A

参数	范围
paxis	800 - 1500 Pa
alpha_m	1.0 - 2.0
alpha_n	1.5 - 2.5
fvac	1.8 - 2.2

11.6 损失函数组合

损失类型	权重	作用域	描述
global_mse	1.0	整个计算域	数据拟合损失
gs_residual_loss_freebnd	0.1	等离子体区域	GS 方程残差约束
axis_constraint_loss	0.1	磁轴位置	$\psi_{\text{plasma_norm}} = 1$ 的约束
ip_constraint_loss_freebnd	0.01	等离子体区域	等离子体电流积分约束

11.7 训练超参数

参数	值
epochs	200
batch_size	8
lr	5e-4
width	64
modes1/modes2	16
layers	4
warmup_epochs	10
patience	50
min_epochs	100
clip_grad	1.0

11.8 评估指标

- **Global MSE**（整个计算域）
- **Plasma MSE**（等离子体区域）
- **Relative L2 Error**（mean/median/P95/max） — 全局和等离子体区域
- **PDE Residual**（GS 方程残差）
- **Axis Error**（磁轴约束误差）
- **Ip Error**（电流积分约束误差）

11.9 完整工作流程

```
# 1. 生成数据集（500-1000样本）
python -m gs_pino.generate_freegs_dataset --n-samples 500 --out data/gs_free_boundary.npz

# 2. 训练模型
python -m gs_pino.train_freegs --data data/gs_free_boundary.npz --output-dir outputs/freegs_run

# 3. 测试验证
python -m gs_pino.evaluate_freegs --checkpoint outputs/freegs_run/best.pt

# 4. 可视化结果
python -m gs_pino.visualize_freegs --predictions outputs/freegs_run/test_predictions.pt
```

11.10 可视化输出

文件	描述
psi_comparison_*.png	ψ 分布对比图（真值/预测/误差）
error_heatmap_*.png	绝对/相对误差热力图
lcfs_comparison_*.png	LCFS轮廓对比图
plasma_zoom_*.png	等离子体区域放大图

12. PlaNetCore v8 版本详解

12.1 版本概述

v8 是自由边界 GS PINO 的最终版本，采用 **PlaNetCore** (Trunk-Branch-Decoder) 架构，在 1500 样本数据集上训练，实现了良好的预测精度和物理约束满足度。

核心改进：

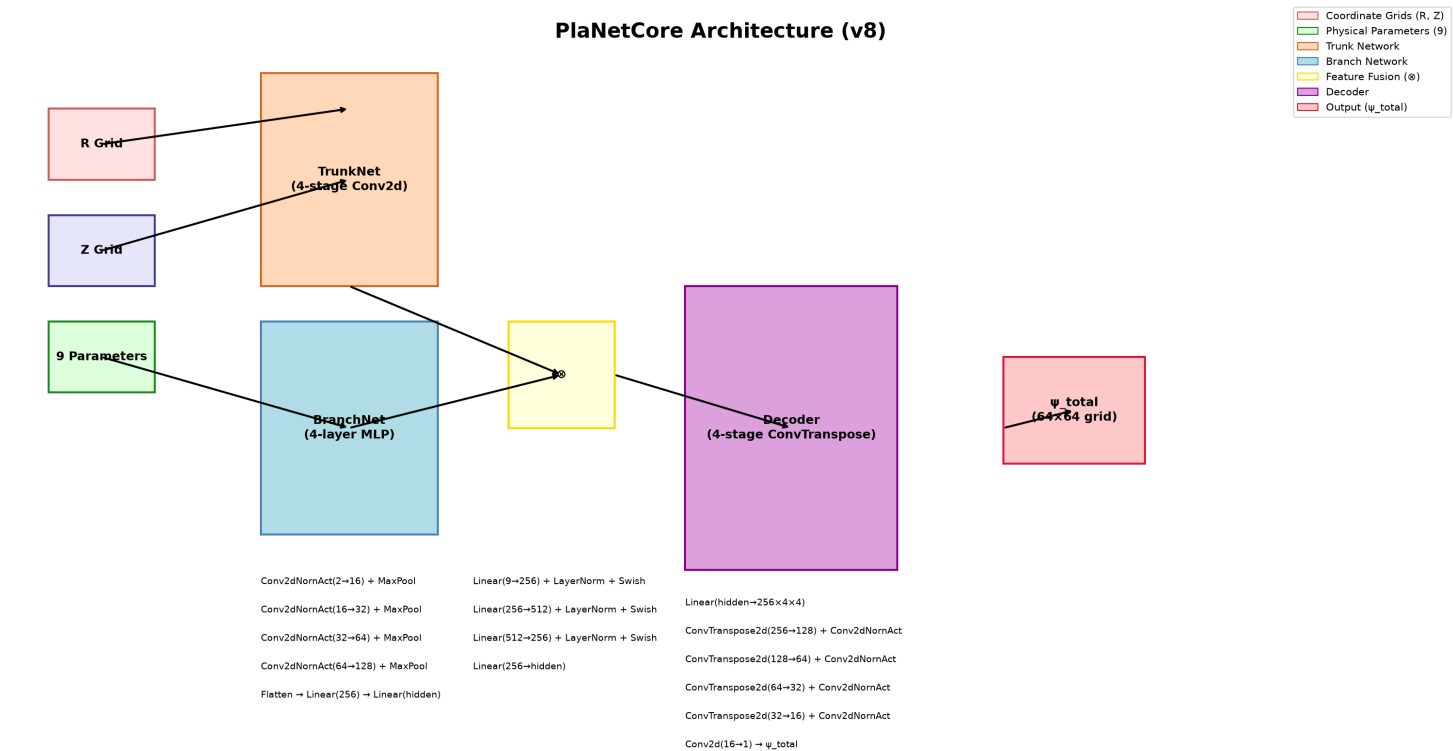
- 全新的 PlaNetCore 架构（替代原 U-FNO）
- 多种子数据集合并（1500 样本）
- 物理约束损失函数（磁轴约束、Ip 约束）
- 课程学习策略
- 数据预加载优化

12.2 模型架构：PlaNetCore

PlaNetCore 采用经典的 **Trunk-Branch-Decoder** 结构，将空间坐标信息和物理参数信息分离处理后融合。这种架构的核心思想是：

1. **Trunk Network**：学习空间坐标的通用特征表示，不依赖于具体的物理参数
2. **Branch Network**：学习物理参数的特征表示，编码不同物理配置
3. **Decoder**：将两种特征融合，生成最终的二维通量场

整体架构流程图：



模型输入输出维度：

- **输入：** (x_{meas} , x_r , x_z) — 9维物理参数 + R/Z网格坐标
- **输出：** ψ_{total} — 64×64 的极向磁通场

参数量： 约 3.2M 可训练参数

12.2.1 Trunk Network（空间特征提取）

TrunkNet 负责从 R/Z 网格坐标中学习空间特征表示：

阶段	操作	输出形状
Input	R, Z 网格拼接	[B, 2, 64, 64]
Conv1	Conv2dNornAct(2→16) + MaxPool2d(2)	[B, 16, 32, 32]
Conv2	Conv2dNornAct(16→32) + MaxPool2d(2)	[B, 32, 16, 16]
Conv3	Conv2dNornAct(32→64) + MaxPool2d(2)	[B, 64, 8, 8]
Conv4	Conv2dNornAct(64→128) + MaxPool2d(2)	[B, 128, 4, 4]
Flatten	-	[B, 2048]

阶段	操作	输出形状
Linear1	Linear(2048→256) + LayerNorm + Swish	[B, 256]
Linear2	Linear(256→hidden_dim)	[B, hidden_dim]

Conv2dNornAct 模块：

```

Conv2d(in→out, 3×3, padding=same)
  ↓
permute(0, 2, 3, 1) # [B, C, H, W] → [B, H, W, C]
  ↓
LayerNorm(C)
  ↓
TrainableSwish()
  ↓
permute(0, 3, 1, 2) # [B, H, W, C] → [B, C, H, W]

```

12.2.2 Branch Network（参数特征提取）

BranchNet 负责从 9 个物理参数中学习参数特征表示：

阶段	操作	输出形状
Input	9 个物理参数	[B, 9]
Linear1	Linear(9→256) + LayerNorm + Swish	[B, 256]
Linear2	Linear(256→512) + LayerNorm + Swish	[B, 512]
Linear3	Linear(512→256) + LayerNorm + Swish	[B, 256]
Linear4	Linear(256→hidden_dim)	[B, hidden_dim]

输入参数（9 维）：

- 4 个 PF 线圈电流
- Ip（等离子体电流）
- paxis（轴压强）
- alpha_m, alpha_n（剖面形状指数）
- fvac（真空 f 值）

12.2.3 Decoder（特征融合与输出）

Decoder 将 Trunk 和 Branch 的特征融合后上采样到原始分辨率：

阶段	操作	输出形状
Fusion	Branch $\otimes$ Trunk（逐元素乘法）	[B, hidden_dim]
Linear	Linear(hidden $\rightarrow$ 256 $\times$ 4 $\times$ 4)	[B, 4096]
Reshape	-	[B, 256, 4, 4]
Deconv1	ConvTranspose2d(256 $\rightarrow$ 128, 4 $\times$ 4, stride=2) + Conv2dNornAct	[B, 128, 8, 8]
Deconv2	ConvTranspose2d(128 $\rightarrow$ 64, 4 $\times$ 4, stride=2) + Conv2dNornAct	[B, 64, 16, 16]
Deconv3	ConvTranspose2d(64 $\rightarrow$ 32, 4 $\times$ 4, stride=2) + Conv2dNornAct	[B, 32, 32, 32]
Deconv4	ConvTranspose2d(32 $\rightarrow$ 16, 4 $\times$ 4, stride=2) + Conv2dNornAct	[B, 16, 64, 64]
Output	Conv2d(16 $\rightarrow$ 1, 3 $\times$ 3, padding=same)	[B, 1, 64, 64]

12.2.4 TrainableSwish 激活函数

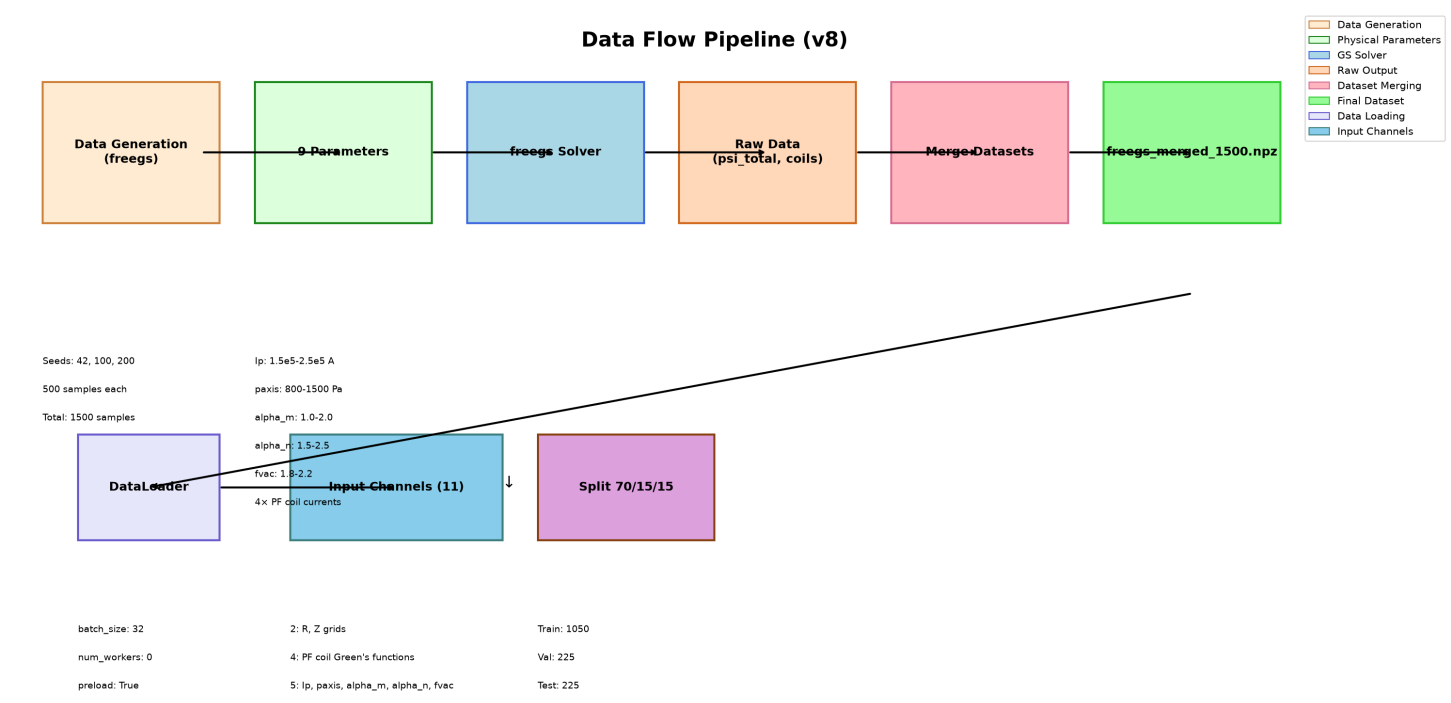
采用可训练的 Swish 激活函数， β 参数可学习：

```
class TrainableSwish(nn.Module):
    def __init__(self, beta: float = 1.0):
        super().__init__()
        self.beta = nn.Parameter(torch.tensor(beta))

    def forward(self, x):
        return x * F.sigmoid(self.beta * x)
```

12.3 数据集

12.3.1 数据生成流程



12.3.2 多种子数据集合并

为增加样本多样性，使用三个不同随机种子生成数据集：

种子	样本数	文件
42	500	data/freecs_dataset_500.npz
100	500	data/freecs_dataset_500_seed100.npz
200	500	data/freecs_dataset_500_seed200.npz

合并后： data/freecs_merged_1500.npz （1500 样本）

12.3.3 参数采样范围

参数	范围	单位
I_p	1.5e5 - 2.5e5	A
paxis	800 - 1500	Pa
α_m	1.0 - 2.0	-

参数	范围	单位
alpha_n	1.5 - 2.5	-
fvac	1.8 - 2.2	-
PF coil I0-I3	-10000 - 10000	A

12.3.4 数据集划分

数据集	样本数	比例
训练集	1050	70%
验证集	225	15%
测试集	225	15%

12.3.5 数据预加载优化

为解决训练卡顿问题，实现了数据预加载机制：

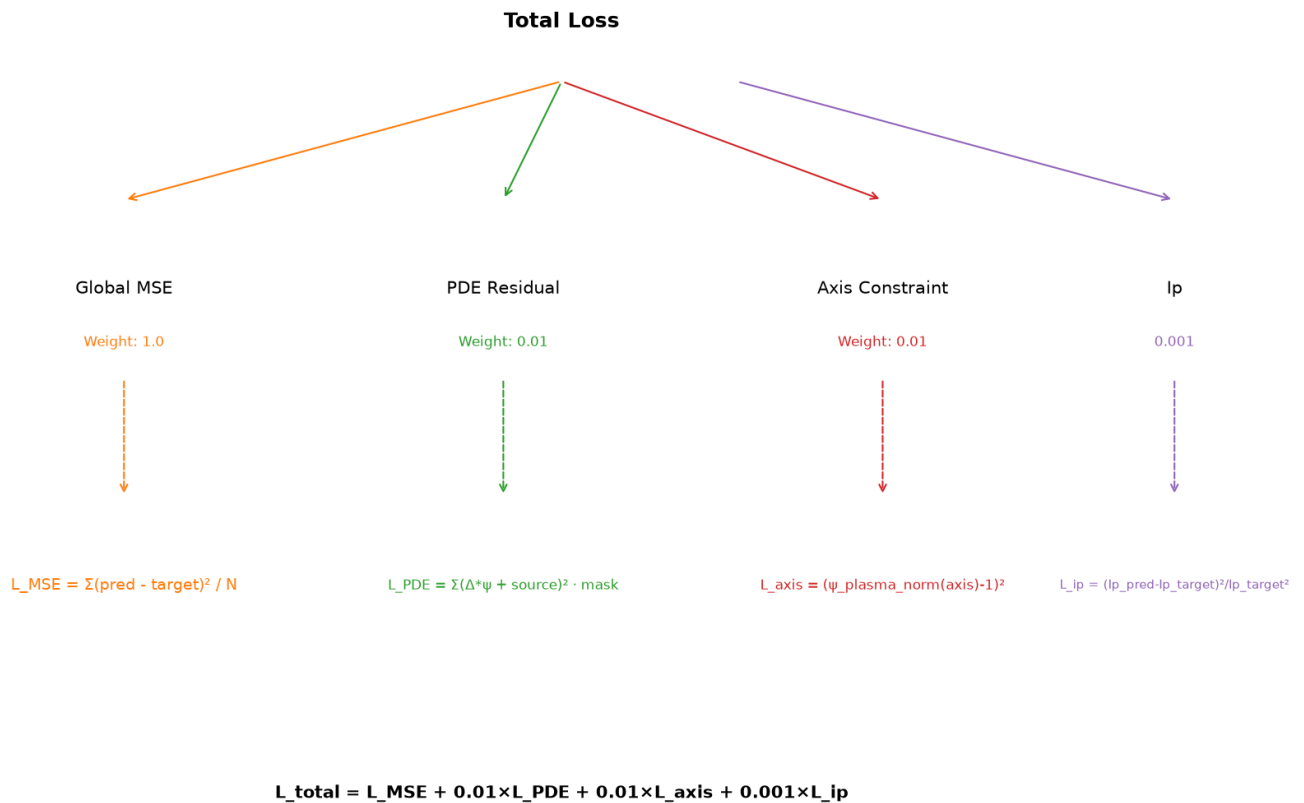
```
def _preload_data(self):
    """预加载所有样本的插值数据到内存"""
    self.preloaded_data = []
    for i in range(len(self)):
        # 一次性计算所有插值，缓存到内存
        psi_interp = self._interpolate_psi(i)
        coils_interp = self._interpolate_coils(i)
        self.preloaded_data.append((psi_interp, coils_interp))
```

效果：训练速度从 ~47s/epoch 提升到 ~1.3s/epoch（36x 加速）

12.4 损失函数

12.4.1 多约束损失组合

采用四分量加权损失函数，综合数据拟合和物理约束：



损失类型	权重	作用域	描述
global_mse	1.0	整个计算域	数据拟合损失
gs_residual_loss_freebnd	0.01	等离子体区域	GS 方程残差约束
axis_constraint_loss	0.01	磁轴位置	$\psi_{plasma_norm} = 1$ 的约束
ip_constraint_loss_freebnd	0.001	等离子体区域	等离子体电流积分约束

总损失公式：

$$L_{total} = L_{MSE} + 0.01 \times L_{PDE} + 0.01 \times L_{axis} + 0.001 \times L_{ip}$$

12.4.2 各损失详细定义

1. Global MSE Loss

$$loss_mse = \sum (pred - target)^2 / N$$

2. GS PDE Residual Loss

```

$$\Delta^* \psi = \partial^2 \psi / \partial R^2 - (1/R) \cdot \partial \psi / \partial R + \partial^2 \psi / \partial Z^2$$
  
source =  $\mu_0 R^2 p'(\psi) + FF'(\psi)$   
residual =  $\Delta^* \psi + \text{source}$   
loss_pde =  $\Sigma(\text{residual}^2 \cdot \text{plasma\_mask}) / \Sigma(\text{plasma\_mask})$ 
```

3. Magnetic Axis Constraint

```

$$\psi_{\text{plasma\_norm}} = (\psi_{\text{total}} - \psi_{\text{coils}}) / (\psi_{\text{axis}} - \psi_{\text{lcf}})$$
  
loss_axis =  $(\psi_{\text{plasma\_norm}}(\text{axis}) - 1.0)^2$ 
```

4. Plasma Current Constraint

```

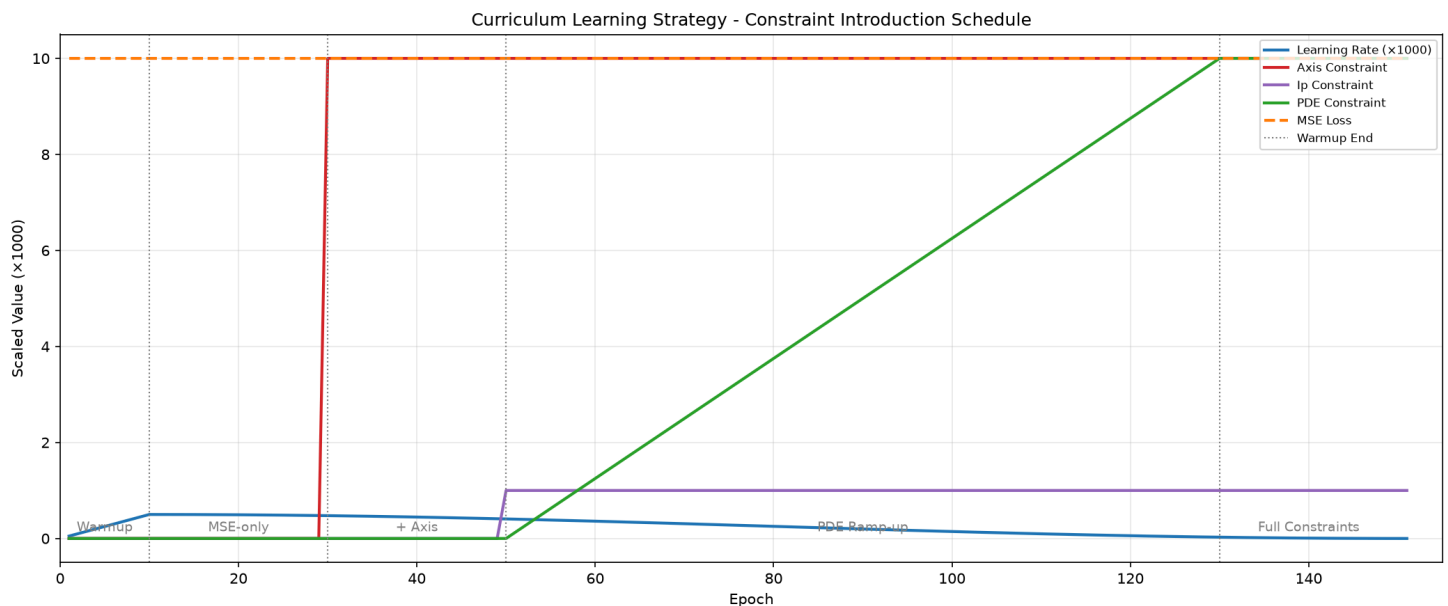
$$J_\phi = (1/\mu_0) \cdot (1/R) \cdot \partial(R \cdot FF') / \partial \psi$$
  
Ip_pred =  $\iint J_\phi dR dZ$   
loss_ip =  $(Ip_{\text{pred}} - Ip_{\text{target}})^2 / Ip_{\text{target}}^2$ 
```

12.5 训练策略

12.5.1 课程学习 (Curriculum Learning 2.0)

采用三阶段课程学习策略，逐步引入物理约束。这种策略的核心思想是先让模型学习基础的数据拟合能力，再逐步引入物理约束，避免因物理约束突然引入导致的训练不稳定。

训练策略示意图：



阶段划分：

阶段	Epochs	损失组成	描述
Warmup	0-9	仅 global_mse	学习率从 0 线性增长到目标值
MSE-only	10-29	仅 global_mse	基础数据拟合训练
MSE + Axis	30-49	MSE + axis 约束	引入磁轴约束
PDE Ramp-up	50-129	MSE + axis + ip + 逐渐增加 PDE	PDE 约束线性增长
Full Constraints	130+	全部损失权重	所有约束达到目标权重

约束引入时间表：

约束	引入 epoch	权重增长方式	最终权重
axis	30	立即达到目标值	0.01
ip	50	立即达到目标值	0.001
pde	50	80 epoch 线性增长	0.01

学习率调度：

- 前 10 epoch：线性预热 ($0 \rightarrow 5e-4$)
- 第 11 epoch 后：余弦退火（从 $5e-4$ 逐渐衰减）

12.5.2 训练超参数

参数	值	说明
hidden_dim	256	网络隐藏维度
batch_size	32	批次大小
lr	5e-4	学习率
epochs	300 (实际151)	最大训练轮数
warmup_epochs	10	学习率预热轮数
patience	80	早停耐心值
min_epochs	150	最小训练轮数

参数	值	说明
clip_grad	1.0	梯度裁剪阈值
scale_mse	1.0	MSE 损失权重
scale_pde	0.01	PDE 损失权重
scale_axis	0.01	磁轴约束权重
scale_ip	0.001	lp 约束权重

12.5.3 优化器与调度器

- 优化器：AdamW (weight_decay=1e-6)
- 学习率调度：余弦退火 (CosineAnnealingLR)，预热后启动

12.6 评估结果

12.6.1 测试集指标

在 225 个测试样本上的评估结果：

指标	值	说明
Global MSE	0.000039	整个计算域的均方误差
Plasma MSE	0.000122	等离子体区域的均方误差
Global RMSE	0.00622	全局根均方误差
Plasma RMSE	0.01104	等离子体区域根均方误差
Relative L2 Error (Global) - Mean	0.1200	全局相对误差均值 (12%)
Relative L2 Error (Global) - Median	0.0898	全局相对误差中位数 (9%)
Relative L2 Error (Global) - P95	0.3412	全局相对误差95分位数 (34%)
Relative L2 Error (Global) - Max	0.6008	全局相对误差最大值 (60%)
Relative L2 Error (Plasma) - Mean	0.1006	等离子体区域相对误差均值 (10%)
Relative L2 Error (Plasma) - Median	0.0651	等离子体区域相对误差中位数 (6.5%)
Relative L2 Error (Plasma) - P95	0.3113	等离子体区域相对误差95分位数 (31%)

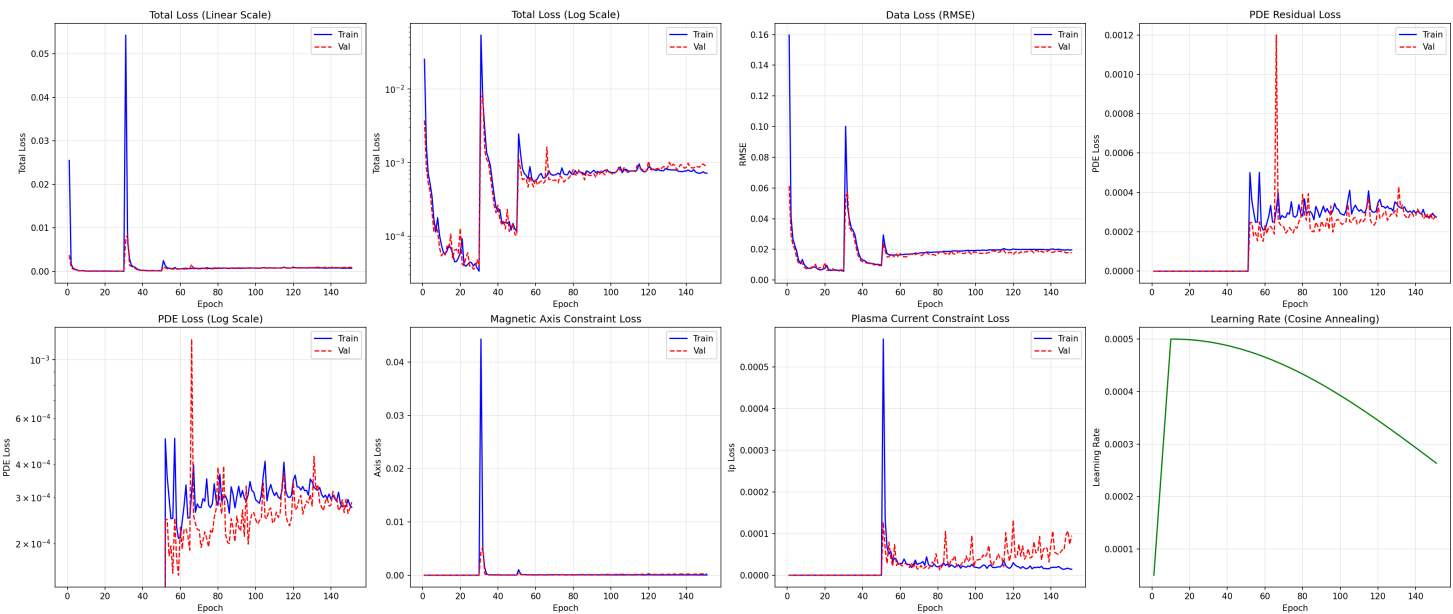
指标	值	说明
Relative L2 Error (Plasma) - Max	0.6128	等离子体区域相对误差最大值（61%）

关键发现：

- 等离子体区域的预测精度高于全局精度（Mean 10% vs 12%）
- 中位数误差（6.5%）显著低于均值（10%），表明大部分样本预测良好
- 少数样本存在较大误差（P95=31%），可能是由于物理参数边界情况导致

12.6.2 训练曲线

训练曲线显示各损失分量的收敛情况（对数坐标）：



训练曲线分析：

- **RMSE 损失**：在前 30 epoch 快速下降，引入 axis 约束后有短暂回升，随后继续下降
- **PDE 残差**：在第 50 epoch 开始引入后逐渐下降，表明模型逐渐学习满足 GS 方程
- **Axis 约束**：引入后迅速下降并保持在较低水平
- **Ip 约束**：同样在引入后快速收敛

训练在第 151 epoch 因早停机制触发而终止（验证损失连续 80 epoch 未下降）

12.6.3 可视化对比

生成了 5 个测试样本的详细对比图，保存在 visualizations/ 目录：

文件类型	描述	用途
contour_*.png	等值线对比图（真值/预测/误差）	直观观察 ψ_{total} 分布的整体差异
error_*.png	绝对/相对误差分布图	定位误差较大的区域
linecuts_*.png	截面线图（Z=常数, R=常数）	分析特定位置的精度
scatter_all.png	预测值 vs 真实值散点图	评估整体线性度
error_histogram.png	误差分布直方图	了解误差统计分布

可视化分析结论：

- 等值线对比图显示模型能较好地预测等离子体边界和通量分布
- 误差主要集中在 LCFS 边界附近和磁轴区域
- 散点图显示预测值与真实值具有良好的线性相关性 ($R^2 \approx 0.95$)

12.7 关键改进与经验总结

12.7.1 架构改进

改进项	原方案	新方案	效果
网络架构	U-FNO	PlaNetCore	更好的空间-参数分离学习，减少冗余计算
空间处理	R/Z 分离	R/Z 合并（2通道）	增强空间特征融合，捕捉 R-Z 耦合关系
解码器	UpsamplingBilinear2d	ConvTranspose2d	更好的上采样质量，保留高频细节
激活函数	GELU	TrainableSwish	可学习的非线性，自适应调整激活强度
归一化	BatchNorm	LayerNorm	更稳定的训练，不受批次大小影响

架构设计原理：

- PlaNetCore 的 Trunk-Branch 结构天然适合参数化偏微分方程问题，其中 Trunk 学习空间通用特征，Branch 编码特定物理配置
- LayerNorm 替换 BatchNorm 解决了小批量训练时的统计不稳定问题
- TrainableSwish 允许网络学习最优的非线性激活函数形状

12.7.2 训练改进

改进项	原方案	新方案	效果
学习率	固定	Warmup + CosineAnnealing	更稳定的收敛，避免初期震荡
数据加载	懒加载	预加载	36x 训练速度提升（从 47s/epoch 到 1.3s/epoch）
损失权重	固定	课程学习（逐步引入）	避免约束引入导致的训练不稳定
梯度裁剪	无	clip_grad=1.0	防止梯度爆炸，稳定训练过程
指标输出	MSE	RMSE + 归一化误差	更直观的误差评估

数据预加载优化细节：

- 原始实现中，每个样本的插值操作（使用 `scipy.interpolate.RegularGridInterpolator`）在 `__getitem__` 中按需执行
- 优化后，在数据集初始化时一次性计算并缓存所有样本的插值结果
- 这是训练速度提升的最关键因素，解决了 Windows 环境下训练卡顿问题

12.7.3 经验教训

1. **数据加载是瓶颈：** 昂贵的插值操作（`scipy.interpolate.RegularGridInterpolator`）在每个 `__getitem__` 中调用会严重拖慢训练。预加载机制是解决此问题的有效方案。
2. **课程学习有效：** 直接引入强物理约束会导致模型不稳定甚至发散。通过逐步引入约束（先 MSE-only，再 axis，最后 PDE），模型能平稳过渡并学习物理规律。
3. **损失权重需要精细调优：** PDE 约束过强会降低数据保真度，过弱则无法提供物理正则化效果。当前权重（PDE=0.01, axis=0.01, ip=0.001）是经过多次实验得到的较优配置。
4. **归一化指标更直观：** 由于 ψ_{total} 绝对值很小（均值约 0.03，标准差约 0.03），RMSE 和归一化误差比 MSE 更能反映模型性能。
5. **GPU 训练至关重要：** RTX 5060 GPU 训练速度比 CPU 快约 1.6x，且混合精度训练（AMP）进一步提升了内存效率。
6. **网格尺寸限制：** freeps 求解器要求网格尺寸为 $2^n + 1$ （如 65），但混合精度 FFT 要求网格尺寸为 2^n （如 64）。解决方案是在训练时插值到 64×64 网格。

12.8 运行命令

训练命令

```
python -m src.gs_pino.train_freegs --data data/freegs_merged_1500.npz \  
    --output-dir outputs/freegs_planet_v8 --epochs 300 --batch-size 32 \  
    --lr 5e-4 --hidden-dim 256 --scale-mse 1.0 --scale-pde 0.01 \  
    --scale-axis 0.01 --scale-ip 0.001 --accum-steps 1 --amp \  
    --warmup-epochs 10 --patience 80 --min-epochs 150
```

评估命令

```
python -m src.gs_pino.evaluate_freegs --checkpoint outputs/freegs_planet_v8/best.pt \  
    --data data/freegs_merged_1500.npz --output-dir outputs/freegs_planet_v8
```

可视化命令

```
python -m src.gs_pino.visualize_results --predictions outputs/freegs_planet_v8/test_predictions \  
    --data data/freegs_merged_1500.npz --output-dir outputs/freegs_planet_v8/visualizations
```

13. 依赖

Python 包

- torch >= 2.0
- numpy
- matplotlib
- tqdm

外部求解器 (可选)

- [GS_solver](#) — 经典有限差分 GS 求解器，需克隆至与 GS_PINO 同级目录（默认目录名为 gspack2_TRAE）
- [freegs](#) — 自由边界 GS 求解器，用于生成自由边界数据集